计算机科学与技术学院 神经网络与深度学习课程实验报告

实验题目：实验二 回归		学号：202200130077
日期：2024.9.11	班级：22 级人工智能	姓名：薛润泽
Email：		

实验目的：

学会构建线性回归模型以及模型训练，并使用 matplotlib 进行可视化。

实验软件和硬件环境：

Windows、Vscode

实验原理和方法：

基本模型为 y = wx。

原理：当将 x 带入模型 y = wx 时，可以得到 y_pre 的值。通过计算实际值 y 与预测值 y_pre 的差的平方，并将全部(y_pre - y)^2 求和可以得到损失函数 L。

$$L = \Sigma(y_{pre} - y)^2$$

我们通过不断调整 w 的值，来减小损失函数。当最小化损失函数 L 时，我们得到的 w 即为收敛时的 w。

方法：建立损失函数 L 模型，通过使用随机梯度下降法来不断更新 w 的值，直到随机梯度下降法收敛。

实验步骤：（不要求罗列完整源代码）

1、构建出随机的线性点数据

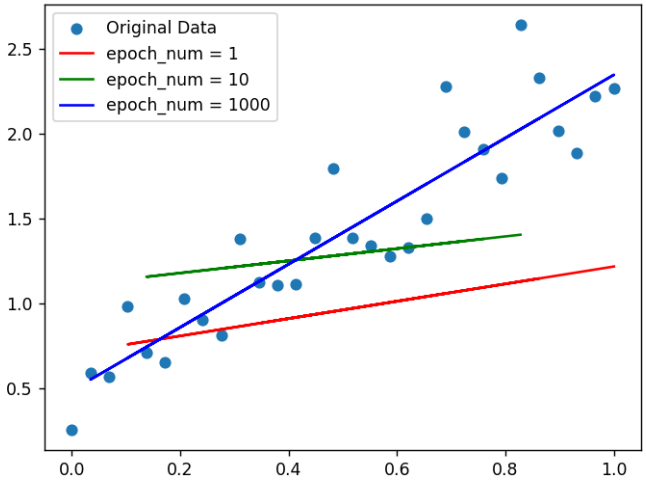
2、建立线性回归模型，确定模型的损失函数，学习率，优化函数

3、分批次训练模型，采用 epoch 和 step 调整模型训练次数。其中 epoch 调整总数据的使用次数，step 调整分几次使用总数据。在本次实验中，使用了不同的 3 种不同的 epoch，分别是 1 次，10 次，1000 次。

4、根据训练出的模型，带入 x，得到预测的 y_pre，并画图

结论分析与体会：

实验结果如下图：



可以看到，随着训练次数的不断增加，回归模型效果越好。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：

Epoch 和 step 是怎样调控模型训练数据的？

通过使用 DataLoader，可以将数据分批次进行训练。

代码附录：

```
import torch
#随机构造线性数据
import math
import matplotlib.pyplot as plt
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
class LinearRegression(nn.Module):
    def __init__(self):
        super(LinearRegression, self).__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)

#清空模型，初始化模型参数
def reset_model_parameters(model):
    for layer in model.children():
        if hasattr(layer, 'reset_parameters'):
```

```
            layer.reset_parameters()


nums_data = 30
x = torch.linspace(0, 1,nums_data)
x = torch.unsqueeze(x,dim=1)
k = 2
y = k * x + torch.rand(x.size())
criterion =  nn.MSELoss()
model = LinearRegression()
learning_rate = 0.01
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)#优化函数

dataset = TensorDataset(x, y)
batch_size = 10
dataloader = DataLoader(dataset, batch_size = batch_size, shuffle = True)

epoch_num = 1
step_num = math.ceil(nums_data / batch_size)
x = torch.linspace(0, 1,nums_data)
plt.scatter(x.numpy(), y.numpy(), label = 'Original Data')

for epoch in range(epoch_num):
    for step in range(step_num):
        for batch_x, batch_y in dataloader:
            y_pre = model(batch_x)
            loss = criterion(y_pre, batch_y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

plt.plot(batch_x.numpy(), y_pre.detach().numpy(), label = 'epoch_num = 1',
color = 'red')

reset_model_parameters(model)
epoch_num = 10
```

```
step_num = math.ceil(nums_data / batch_size)
for epoch in range(epoch_num):
    for step in range(step_num):
        for batch_x, batch_y in dataloader:
            y_pre = model(batch_x)
            loss = criterion(y_pre, batch_y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

plt.plot(batch_x.numpy(), y_pre.detach().numpy(), label = 'epoch_num = 10',
color = 'green')
reset_model_parameters(model)

epoch_num = 1000
step_num = math.ceil(nums_data / batch_size)
for epoch in range(epoch_num):
    for step in range(step_num):
        for batch_x, batch_y in dataloader:
            y_pre = model(batch_x)
            loss = criterion(y_pre, batch_y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

plt.plot(batch_x.numpy(), y_pre.detach().numpy(), label = 'epoch_num =
1000', color = 'blue')

# plt.figure()


# y_pre = model(x)
# plt.plot(x.numpy(), y_pre.detach().numpy(), label = 'Fitted Line2', color
= 'green')
# print(1)
```

```
# plt.scatter(x.numpy(), y.numpy(), label = 'Original Data')
plt.legend()
plt.show()

#是否画图
# if if_plot:
#     plt.scatter(x.numpy(),y.numpy(),c=x.numpy())
#     plt.show()
# data = {"x":x, "y":y}

# print(1222)
```